

Attention to Mamba: A Recipe for Cross-Architecture Distillation

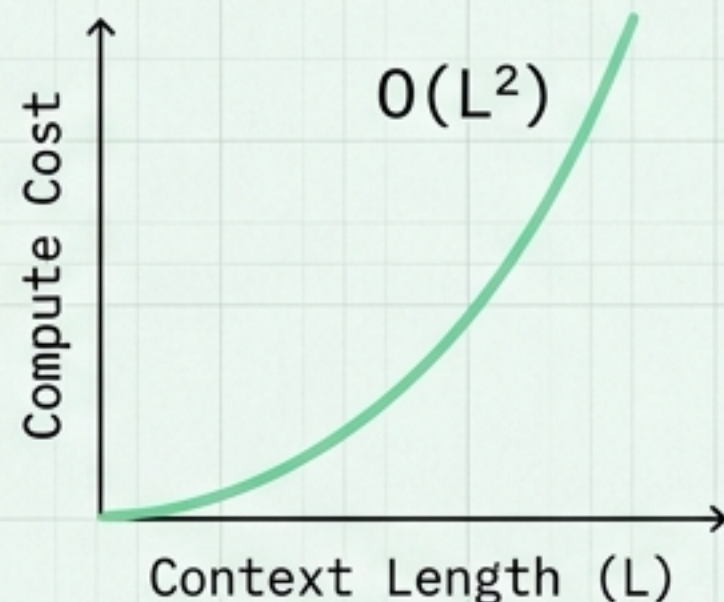
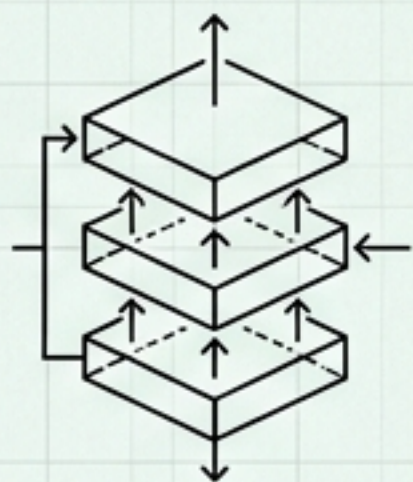
เคล็ดลับการถ่ายทอดความรู้จาก Transformer สู่โมเดล State Space ความเร็วสูง (SSM)

Abhinav Moudgil, Ningyuan Huang, Eeshan Gunesh Dhekane,
Pau Rodríguez, Luca Zappella, Federico Danieli

Apple, MILA, Flat Iron

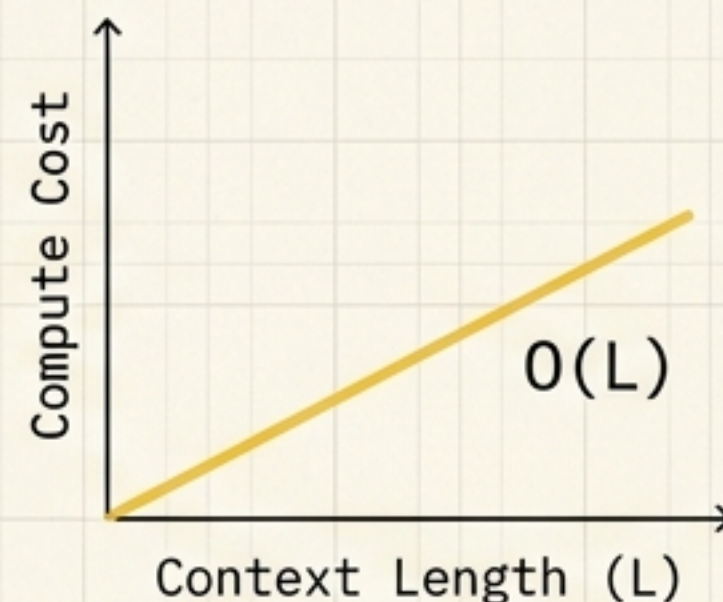
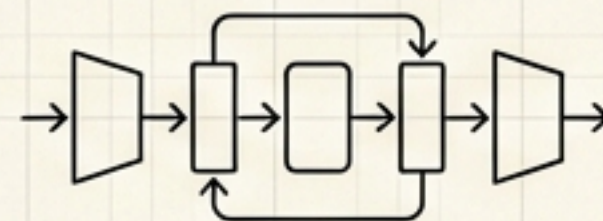
INIT_RECIPE // DOC_01

ความขัดแย้งของสองสถาปัตยกรรม: ประสิทธิภาพ หรือ ความเร็ว?



Transformer

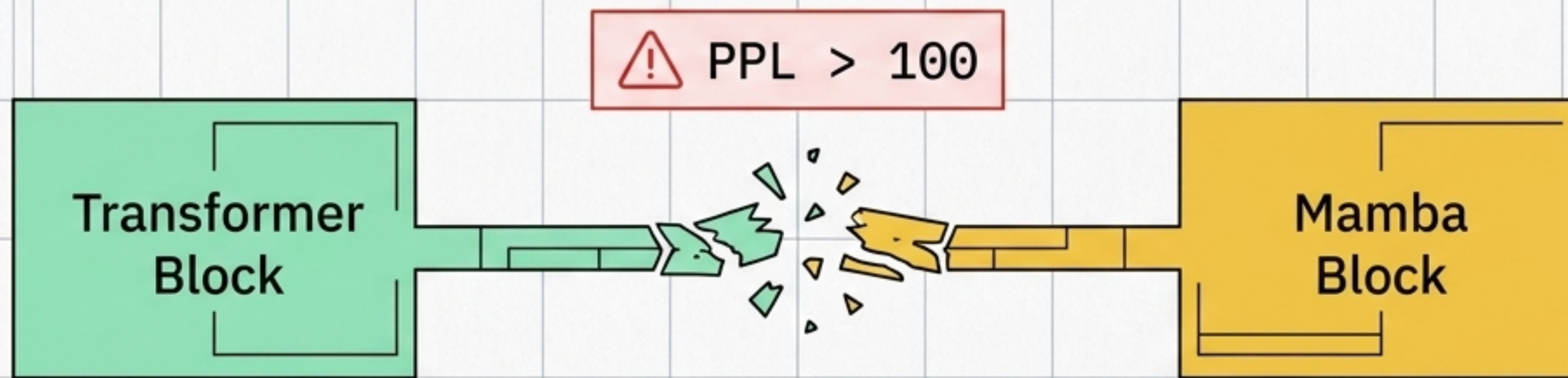
- **จุดเด่น:** ประสิทธิภาพสูง (SOTA), มี Ecosystem ของ Pre-trained โมเดลที่สมบูรณ์
- **คอขวด:** การใช้พลังงานประมวลผลเพิ่มขึ้นแบบทวีคูณ $O(L^2)$ ทำให้มีข้อจำกัดด้าน Context Length



Mamba / SSM

- **จุดเด่น:** อนุมานผล (Inference) รวดเร็ว, ใช้หน่วยความจำต่ำ, การประมวลผลแบบเชิงเส้น $O(L)$
- **คอขวด:** ยังขาดโมเดล Pre-trained ขนาดใหญ่ที่พร้อมใช้งานทันที

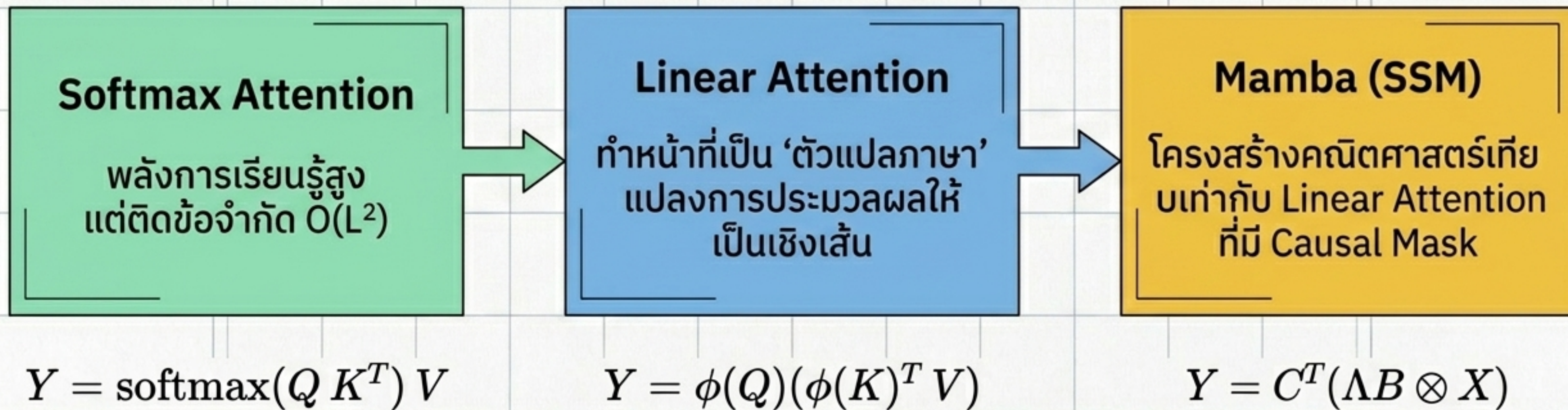
ปัญหา: การทำ Naive Distillation โดยตรงล้มเหลว



- สถาปัตยกรรมทั้งสองพูด ‘คนละภาษาทางคณิตศาสตร์’
- การบังคับถ่ายทอดความรู้ข้ามสถาปัตยกรรม (Cross-architecture Distillation) แบบตรงๆ ทำให้โมเดลสูญเสียความสามารถดั้งเดิม
- ผลลัพธ์: ค่า Perplexity (PPL) พุ่งสูงเกิน 100

เราต้องการ ‘ตัวแปลภาษา’ ที่มีหลักการทางคณิตศาสตร์รองรับ
เพื่อไม่ให้ความรู้จาก Teacher สูญหายระหว่างทาง

กุญแจสำคัญ: คณิตศาสตร์คือสะพานเชื่อม



Insight: หากเราแปลง (Linearize) Attention ก่อน Mamba จะสามารถรับรู้และเข้าใจน้ำหนัก (Weights) ของ Transformer ได้อย่างสมบูรณ์

กระบวนการ 2 ขั้นตอน (The 2-Stage Recipe)

Stage 1: สกัดความรู้สู่ Linear Attention (Hedgehog)

- ฝึกสอนฟีเจอร์แมป (Feature Map) เพื่อจำลองการทำงานของ Softmax ด้วย Kernel Trick
- เป้าหมาย: รักษาความแม่นยำในรูปแบบเชิงเส้น

Stage 2: ถ่ายทอดสู่ Mamba (HedgeMamba)

- นำพารามิเตอร์เชิงเส้นมาใช้เป็นค่าเริ่มต้น (Initialization) ให้กับ Mamba
- Fine-tune โครงสร้างทั้งหมดเพื่อดึงขีดสุดของความสามารถออกมา

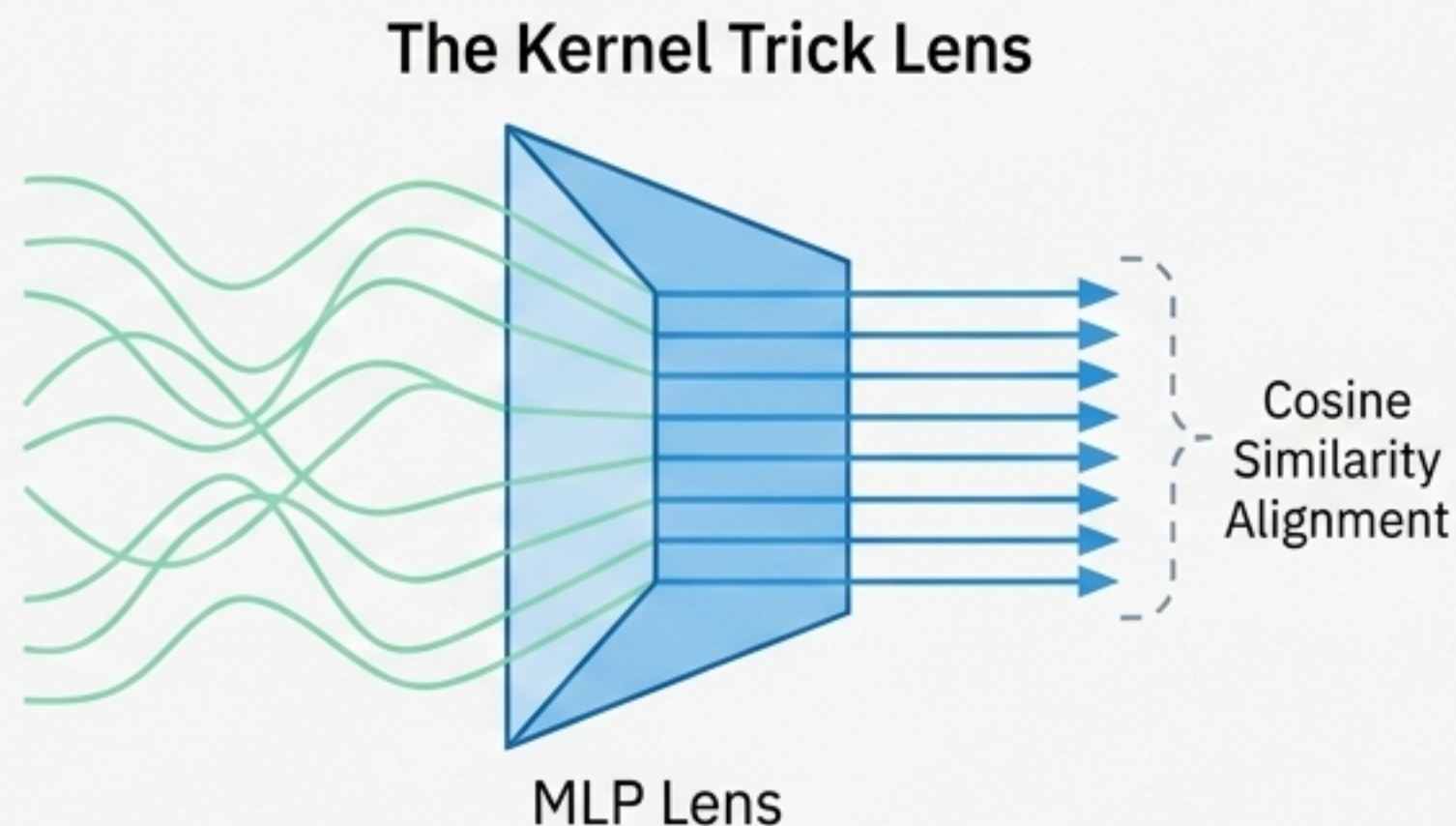
**Output: HedgeMamba
(Hybrid Student)**

เจาะลึก Stage 1: Softmax สู่ Linear (Hedgehog)

Concept: การใช้ Kernel Trick ด้วย MLP (Multi-Layer Perceptron)

Mechanism:

- **ลบข้อจำกัด:** ตัด Nonlinearity (Softmax) ออก
- **ประมาณค่า:** ใช้โมเดล MLP ชั้นเดียว เป็นฟังก์ชันแมปเพื่อเรียนรู้และจำลองพฤติกรรมของ Softmax
- **การปรับเทียบ (Alignment):** ทาบผลลัพธ์ของ Teacher และ Student ด้วย Cosine Similarity Loss

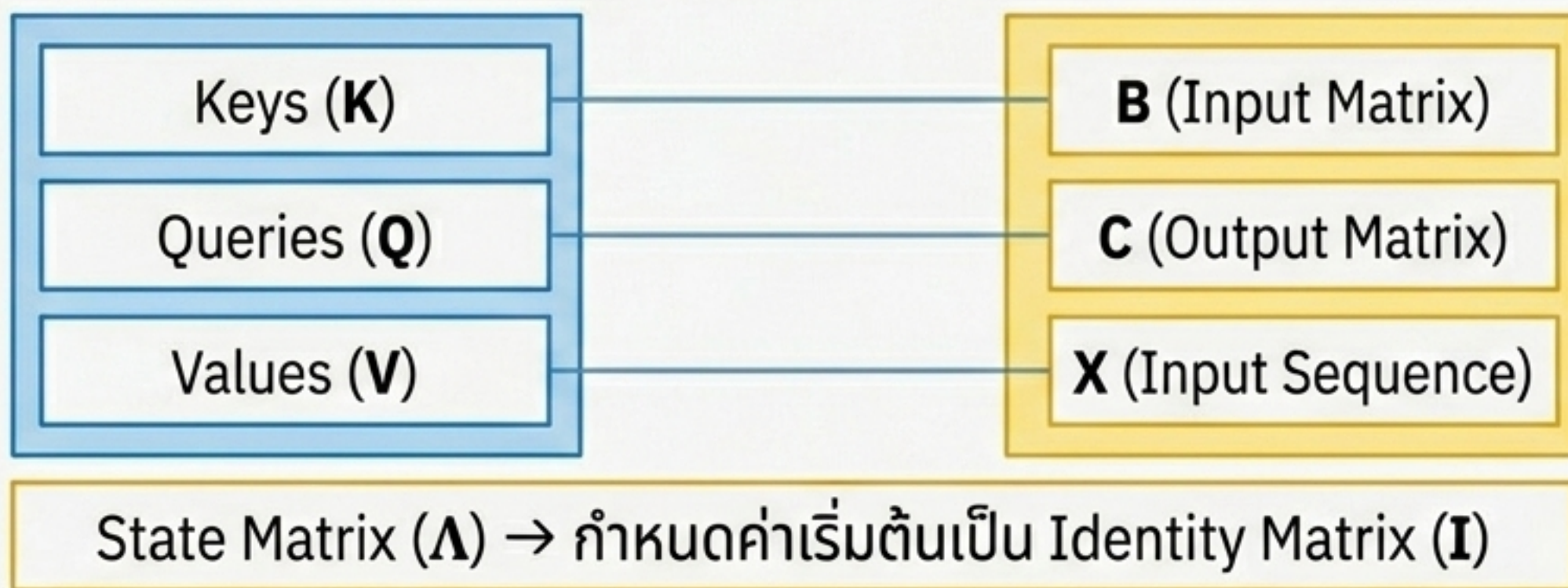


$$e^{\mathbf{x}^T \mathbf{x}'} = \kappa(\mathbf{x}, \mathbf{x}') \approx \phi(\mathbf{x})^T \phi(\mathbf{x}')$$

เจาะลึก Stage 2: การจับคู่พารามิเตอร์สู่ Mamba

Insight: Mamba ไม่ต้องเริ่มเรียนรู้จากศูนย์ เราสามารถนำ Linear Attention มาเสียบแทนที่ (Plug & Play) ได้อย่างลงตัว

Parameter Mapping Diagram



Takeaway: การกำหนดค่าเริ่มต้นที่มีหลักการ (Principled Initialization) คือหัวใจสำคัญที่ทำให้การเรียนรู้ข้ามสถาปัตยกรรมทำงานได้จริง

กายวิภาคของ HedgeMamba

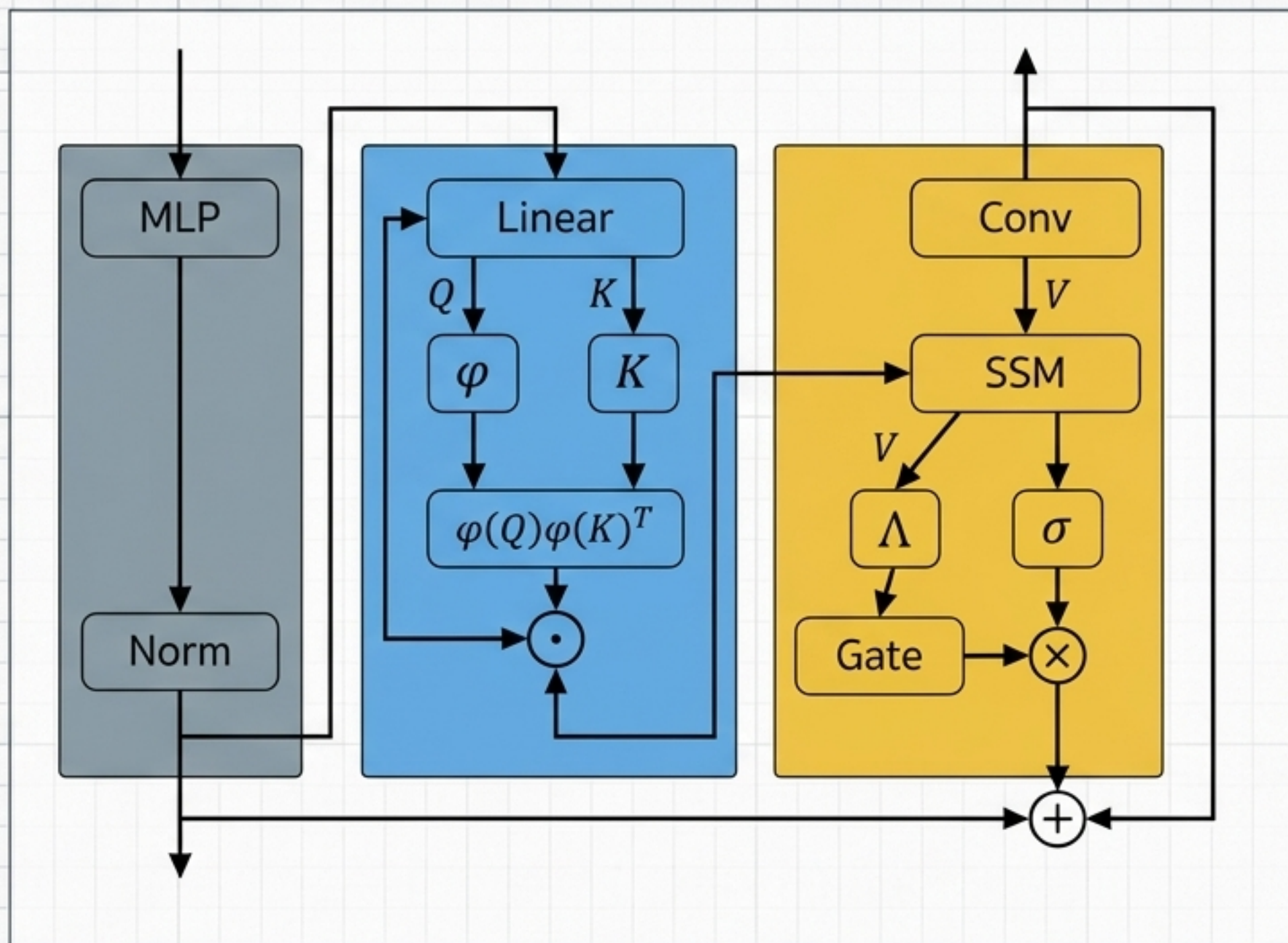
โครงสร้างที่สืบทอดมา (แช่แข็ง / Frozen):

- Norms และ MLP จาก Pythia (Teacher) ถูกคงไว้ตามเดิม

โมดูล Sequence Mixer ใหม่ (Trainable):

- Hedgehog Linear Attention: ทำหน้าที่คำนวณ Query และ Key เชิงเส้น
- Mamba Mixer: รวม Conv, SSM และ Gate สาขาใหม่เข้าด้วยกัน

Outcome: ได้โมเดลแบบไฮบริดที่เข้าใจความรู้สึกของ Transformer แต่ประมวลผลด้วยความเร็วของ Mamba



แดชบอร์ดการทดลอง: บริหารทรัพยากรอย่างมีประสิทธิภาพ

Mission Dashboard

Teacher Model

Pythia-1B
(เทรนด้วยข้อมูล 300B Tokens)

Student Model

HedgeMamba-1B

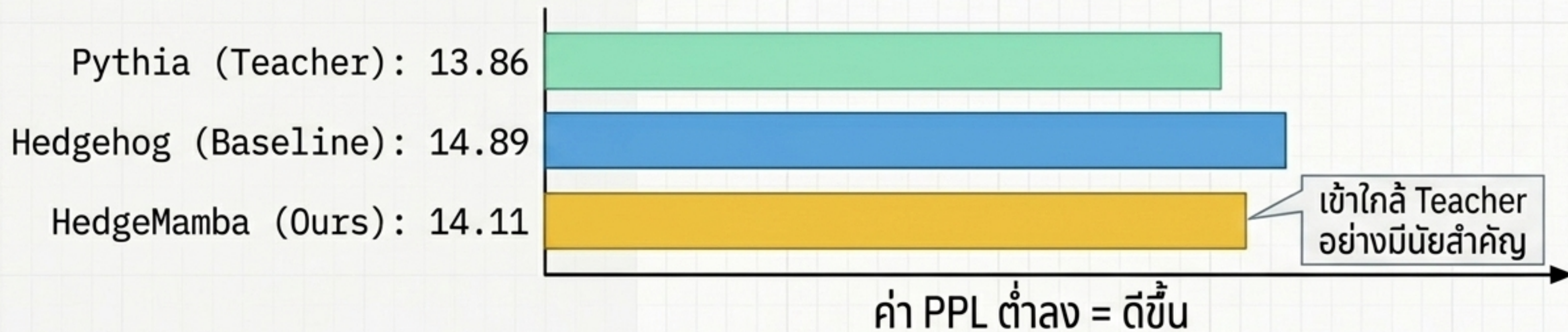
Training Data

OpenWebText
(~9B Tokens)

The Budget Advantage

ใช้งบประมาณผลเพื่อทำ Distillation เพียง 10B Tokens
(เทียบเท่าเพียง ~2.7% ของพลังงานที่ใช้เทรน Teacher ตั้งแต่ต้น!)

ผลลัพธ์ประสิทธิภาพ (Student vs. Teacher)



Zero-shot Downstream Tasks (ความแม่นยำ)

โมเดลสามารถรักษาระดับความสามารถในงานทั่วไปได้อย่างดีเยี่ยม

- ARC-Easy: 53.66%
- HellaSwag: 41.87%

Conclusion

ประสิทธิภาพแทบไม่ตกลงเลย แม้ปรับเปลี่ยนเป็นสถาปัตยกรรมแบบ Linear เชิงเส้นแล้วก็ตาม

ชำแหละสถาปัตยกรรม: พลังของ Gate Branch

คำถาม: อะไรทำให้ HedgeMamba เหนือกว่า Linear Attention ทั้งหมด?

Baseline (Hedgehog)	14.89
+ SSM Parameters (ทรงตัว, ช่วยเตรียมโครงสร้าง)	14.89
+ Convolution Layer (จัดการบริบทระยะสั้น)	14.89
+ Gate Branch (HedgeMamba)	14.58

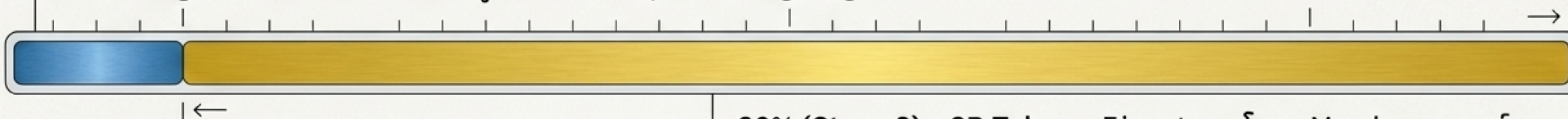
จุดเปลี่ยนสำคัญ!

Key Insight: การเพิ่ม Gate Branch (พร้อม SiLU Non-linearity) ช่วยขยายศักยภาพ (Expressivity) ของโมเดล นำไปสู่การลดลงของ Perplexity อย่างก้าวกระโดด

สัดส่วนทองคำ 10/90: การจัดสรรงบประมาณ Token

Insight: ในการทำ Distillation 2 ขั้นตอน การแบ่ง Token คนละครึ่ง (50/50) ไม่ใช่วิธีที่ดีที่สุด

10% (Stage 1) - 1B Tokens: เรียนรู้ Feature Map ของ Hedgehog

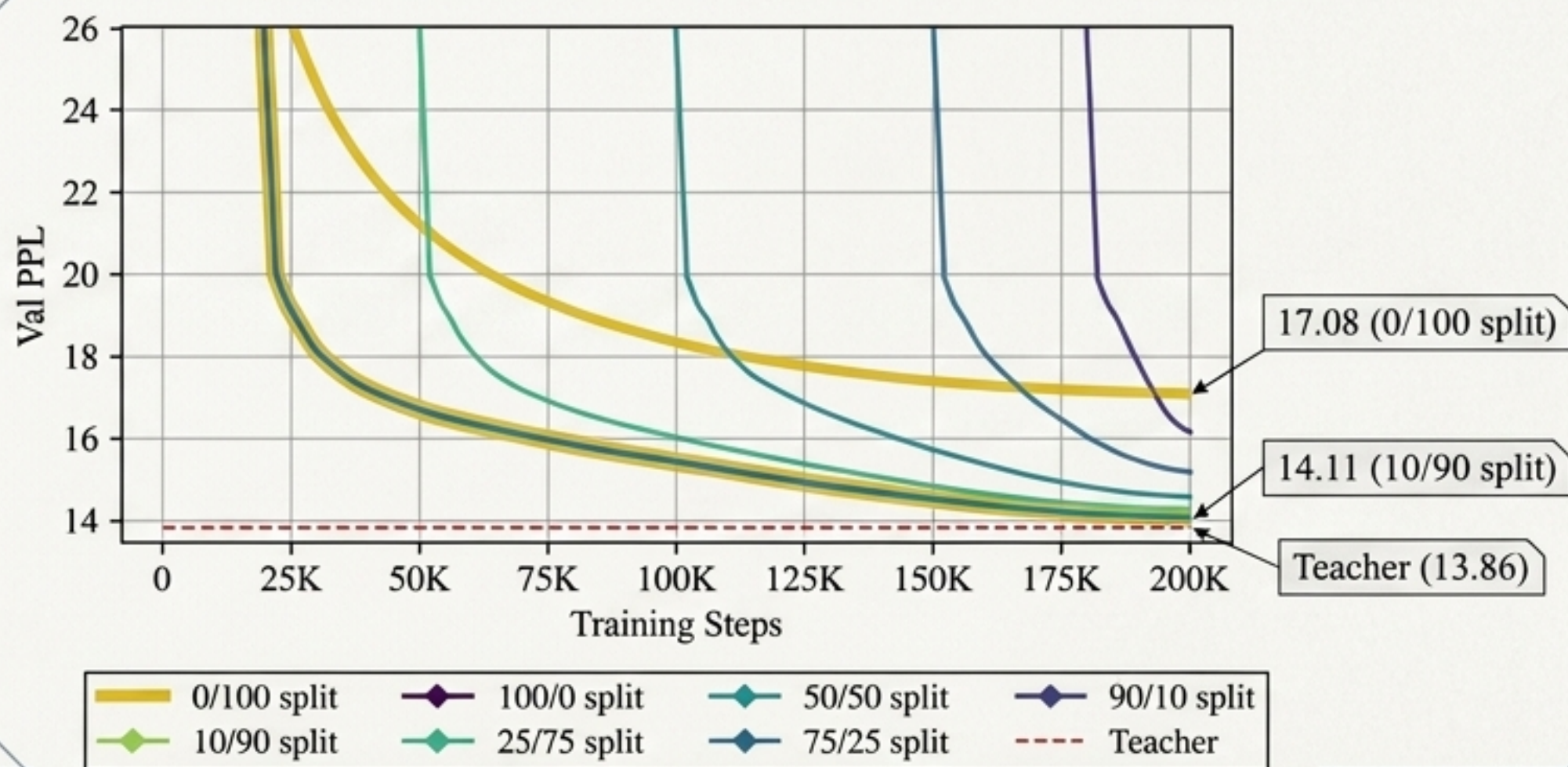


90% (Stage 2) - 9B Tokens: Fine-tune โมเดล Mamba แบบองค์รวม

The Data:

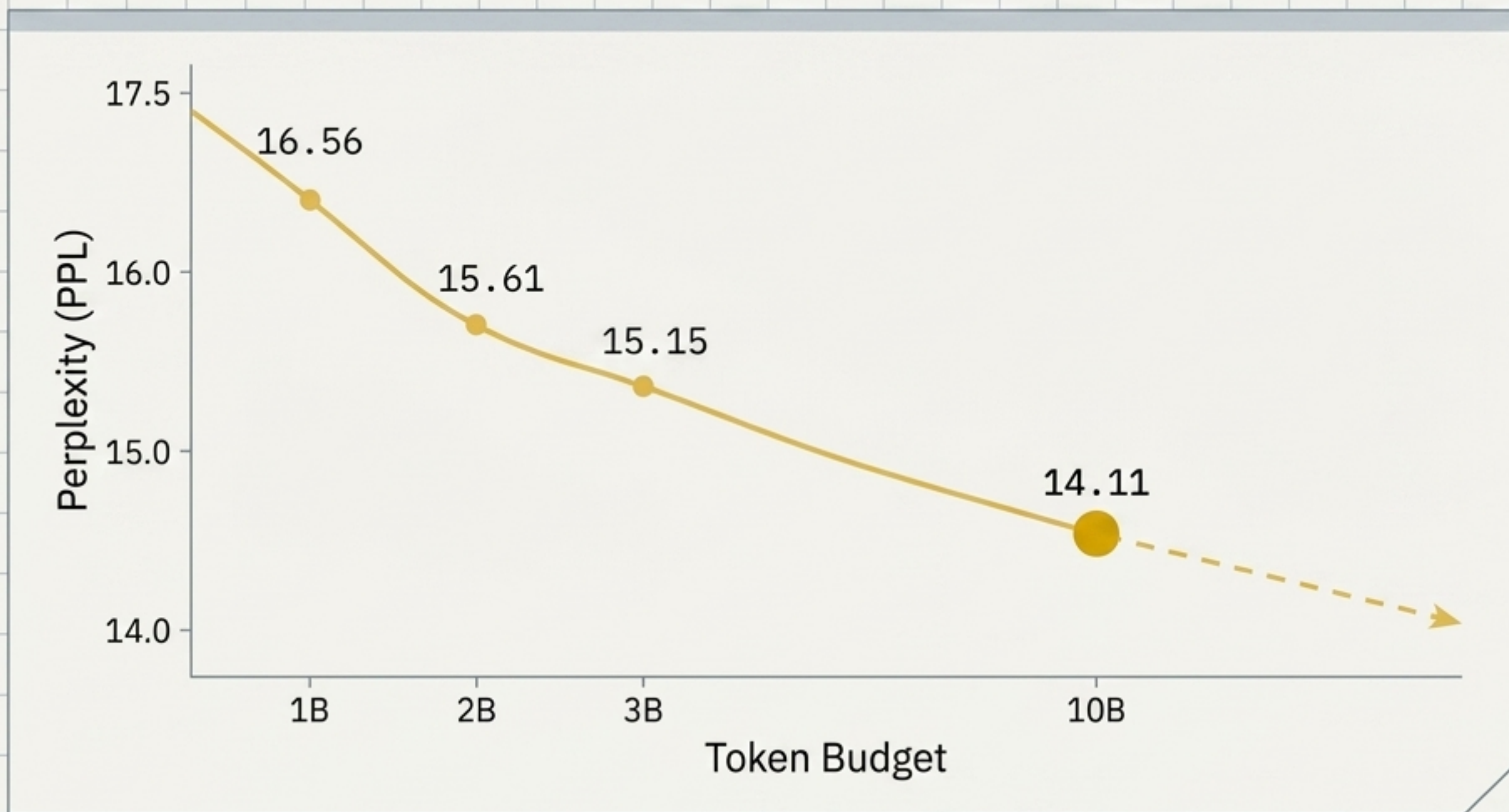
- การจัดสรรแบบ 10/90 ทำค่า PPL ลดลงแตะ 14.11
- เทียบกับการเทรน Stage 2 เพียงอย่างเดียว (0/100) ค่า PPL จะค้างอยู่ที่ 17.08

สรุป: ต้องปูพื้นฐาน Stage 1 เล็กน้อย เสมอ ก่อนเร่งเครื่องใน Stage 2



กฎการขยายขนาด (Scaling Laws) กับ Token Budget

Observation: ประสิทธิภาพของ HedgeMamba จะแปรผันตามจำนวน Distillation Tokens อย่างเป็นระบบ



Future Implication:

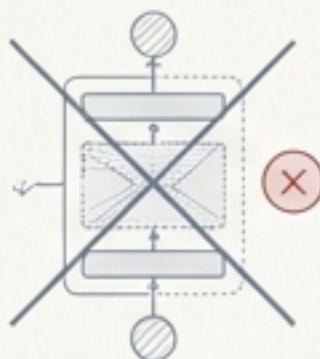
ที่ระดับ 10B Tokens
กราฟการเรียนรู้ยังไม่ถึง
จุดอิ่มตัว (Saturation)
หมายความว่างบประมาณประมวล
ผลที่มากขึ้นจะยิ่งทำให้
โมเดลเก่งขึ้นไปอีก

พิมพ์เขียว: สรุปสมการ Attention to Mamba

The Master Recipe Card

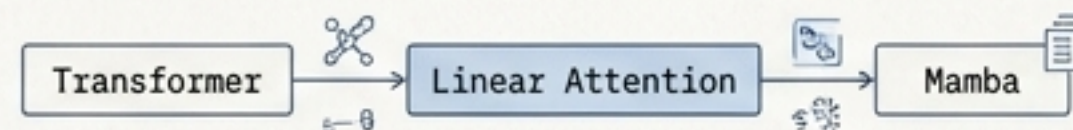
The Problem

Naive Distillation จาก Transformer ไป SSM โดยตรงไม่เวิร์ก (PPL > 100)



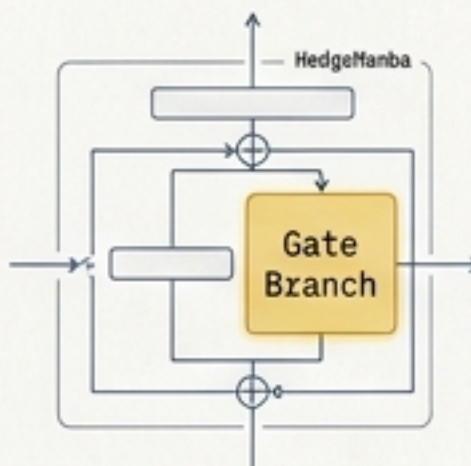
The Solution (2-Stage)

ใช้คณิตศาสตร์สร้างสะพาน:
แปลงเป็น Linear Attention ก่อน
แล้วจึง Initialization สู่ Mamba



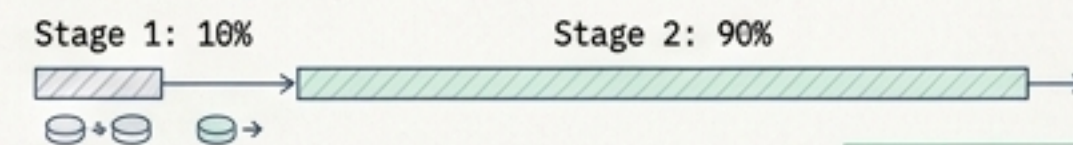
The Architecture

HedgeMamba: ใช้โมดูล Gate Branch เพื่อเพิ่มศักยภาพสูงสุดให้เทียบเท่า Teacher



The Budget Recipe

ใช้ Token เพียง 2.7% ของ Teacher
โดยแบ่งสัดส่วนการเทรนเป็น 10%
(Stage 1) / 90% (Stage 2)



บทสรุปและก้าวต่อไปของวงการ LLM

Key Takeaways:

- เราสามารถปลดล็อกโมเดล Inference ความเร็วสูง (Linear-time) โดยไม่ต้องเสียเงินหลายล้านดอลลาร์เพื่อเทรนโมเดลจากศูนย์
- การกำหนด Initialization ที่มีหลักการทางคณิตศาสตร์
ชนะการทำ Alignment แบบบิบบ้างคับเสมอ

The Future:

ขยายสูตรนี้ไปใช้กับ Transformer ตระกูลอื่นๆ
เพื่อสร้าง Ecosystem ของโมเดลไฮบริดประสิทธิภาพสูง

